현대 강화학습 자율이동체 구현 워크숍

다중 강화학습 기반 무인이동체 제어 구현

Gyu Seon Kim
Korea University, School of Electrical Engineering
Artificial Intelligence and Mobility Laboratory
kingdom0545@korea.ac.kr

목차

강화학습의 기초 및 복습

기초 강화학습 구현

다중 강화학습 기반 무인 이동체 제어

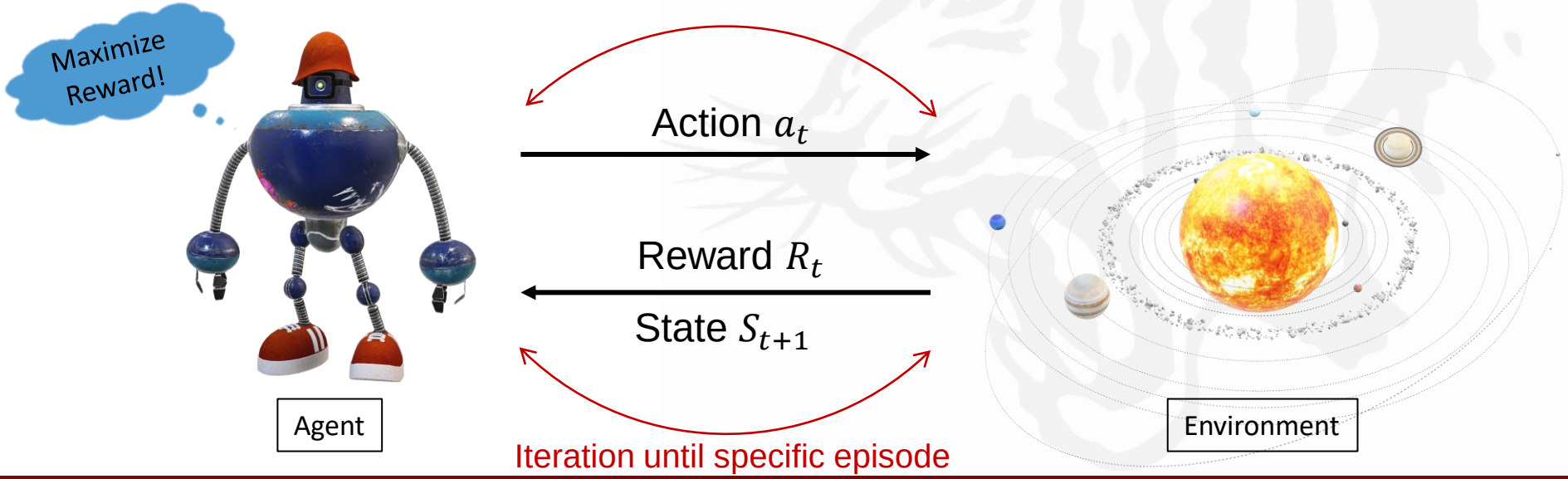
강화학습 기반 제어기 성능평가

무인 이동체 제어 구현

Q&A

➢ 강화학습

- 강화 학습은 에이전트가 Environment (환경)에서 Action (행동)을 수행하고 Reward (보상)의 형태로 피드백을 받아 의사 결정을 내리는 방법을 배우는 일종의 기계 학습
- 에이전트의 목표는 시행착오 과정을 통해 시간이 지남에 따라 **누적 보상을 최대화**하는 정책을 학습하는 것

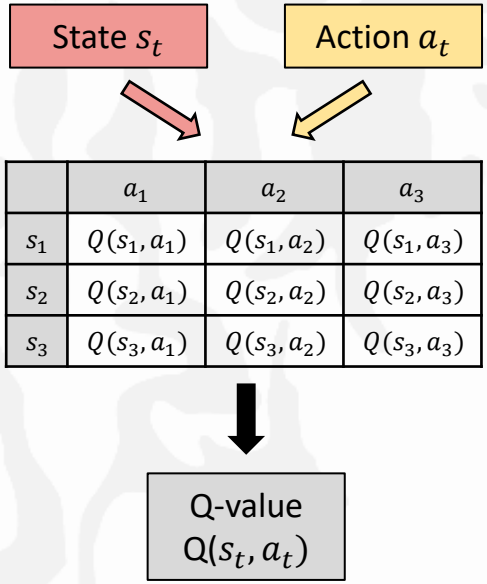


➢ Classic Q-learning

- 현재 상태에서 특정 행동을 취할 때 받는 보상의 기댓값인 Q-value를 학습시켜 최적의 정책을 찾아내는 강화학습 방법
- Q-value를 테이블로 저장하여 학습

$$Q(s,a) \leftarrow Q(s,a) + \alpha(R + \gamma Q(s',a') - Q(s,a))$$

- 타깃 정책과 행동정책이 다른 Off-Policy 학습법
  - ✓ 타깃 정책은 학습의 대상이 되는 정책
  - ✓ 행동 정책은 실제 환경과 상호 작용하며 경험을 쌓고 있는 정책
  - ✓ On-Policy학습이 할 수 없는 과거의 경험 재사용, 일대다/다대일 학습 가능
- 단점
  - ✓ State, Action Space가 커지게 되면 많은 메모리 사용 및 긴 Exploration Time 필요
  - ✓ State Space가 연속적인 경우는 테이블을 만들기 어려움



	a_1	a_2	a_3
s_1	$Q(s_1,a_1)$	$Q(s_1,a_2)$	$Q(s_1,a_3)$
s_2	$Q(s_2,a_1)$	$Q(s_2,a_2)$	$Q(s_2,a_3)$
s_3	$Q(s_3,a_1)$	$Q(s_3,a_2)$	$Q(s_3,a_3)$

<Classic Q-learning>

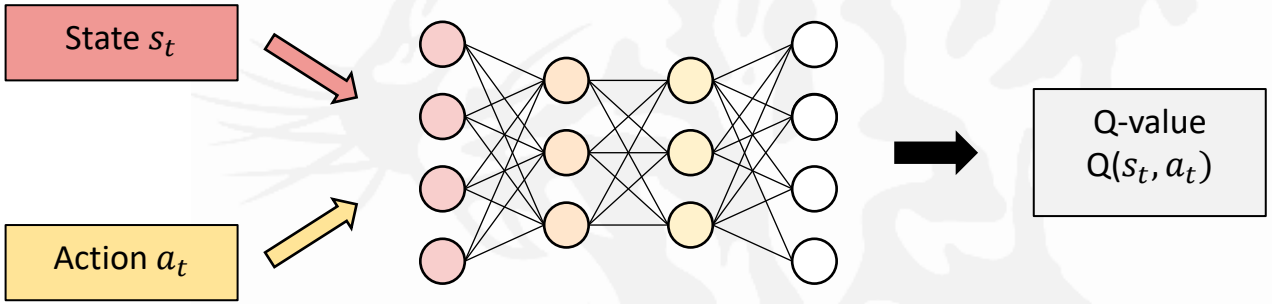
➢ 기존 강화학습에 인공신경망 접목 시도

- 문제점
  - ✓ 학습의 불안정성 문제 발생
  - ✓ 보다 복잡한 상태/행동 공간에서 학습 결과(보상 값)가 수렴하지 않는 문제 발생

- 원인
  - ✓ 학습에 사용되는 데이터 샘플 간의 상관관계
  - ✓ 데이터 분포 변화
  - ✓ 학습이 진행되면서 변화하는 타겟 네트워크

➢ Deep Q-Network

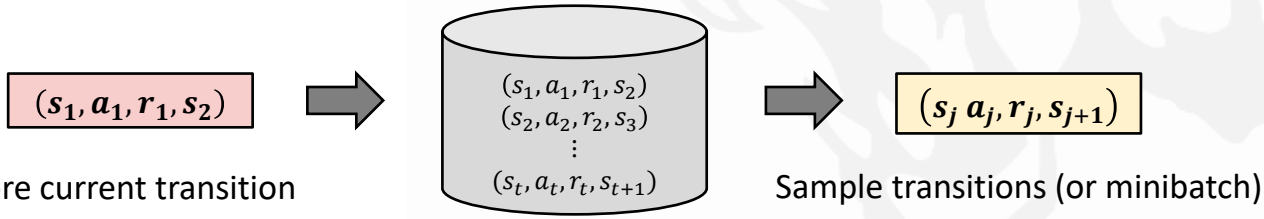
- Q-Learning에 인공신경망(Neural Network) 개념을 도입한 학습법
- 테이블 기반이 아닌 인공신경망을 파라미터 θ를 통해 Q-value를 학습
- Experience Replay와 Target Network의 도입으로 기존 시도의 문제점 해결



<Deep Q-Network>

➢ Experience Replay

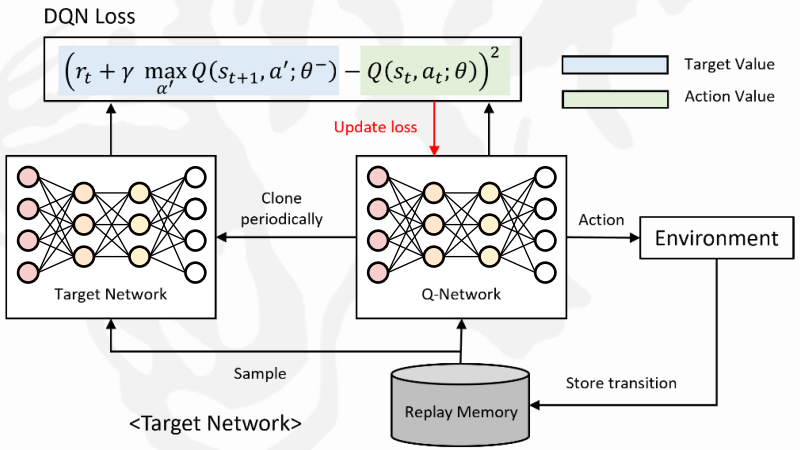
- Replay Memory라는 Buffer을 사용하여, 현재 생성된 샘플 저장
- 현재 선택된 Action을 수행해 결과값과 샘플을 얻지만, 이를 바로 평가에 이용하지 않고 의도적으로 지연시킴
- Replay Memory
  - ✓ 데이터 사이의 상관성을 줄여주기 위해 나온 방법
  - ✓ 가장 최근 t개의 데이터를 Buffer에 넣어줘서 선입선출로 데이터 관리
  - ✓ 학습할 때, 버퍼에서 특정 미니배치 크기만큼 데이터를 랜덤하게 추출
    - → 모든 데이터 여러 번 재사용 가능
  - ✓ 다양한 데이터로 학습하여 데이터 사이의 상관성이 크게 감소
    - → 보다 빠르게 수렴하는 학습 모델 도출



<Replay Memory>

➢ Target Network

- 손실함수를 줄이는 방향으로 θ가 Update
  - → $r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-)$를 Target으로 활용
  - → 안정적인 학습에 문제 발생
- 기존의 Q-Network를 복제하여 Main Q-Network와 Target Network의 이중화된 구조로 구성
- Experience Replay와 Target Network의 도입으로 기존 시도의 문제점 해결
- Target Network
  - ✓ 손실 함수 계산 시, 네트워크 파라미터 잠시 Freeze
    - → 타겟 고정
  - ✓ 특정 주기마다 Freeze된 θ를 새로운 파라미터로 교체
    - → 학습의 안정성 보장



<Target Network>

➢ CartPole-v1

- 카트를 조종하여 막대 제어하는 게임
- **상태(S)**

$$S = \{\text{카트의 위치}, \text{카트의 속력}, \text{막대의 각도}, \text{막대의 각속도}\}$$

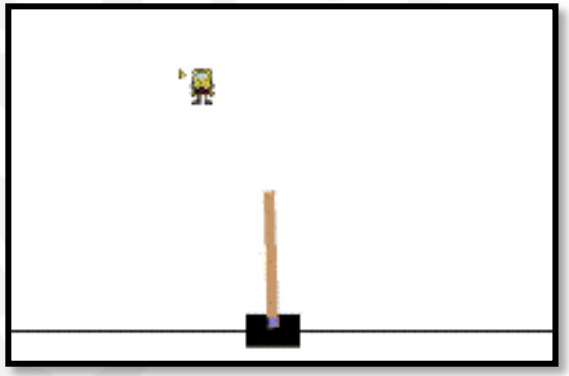
- **행동(A)**

$$A = \{\text{왼쪽으로 이동}, \text{오른쪽으로 이동}\}$$

- **보상(R)**
  - ✓ 중심 기준 일정 범위 내 카트가 존재하고 막대가 넘어지지 않는 경우

$$Reward = +1$$

- **종료**
  - ✓ 막대 각도가 $\pm 12°$보다 큰 경우
  - ✓ 카트 위치가 화면 가장자리에 도달하는 경우
  - ✓ 에피소드가 500번 넘어가는 경우



```python
import gym
import collections
import random
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

```python
#Hyperparameters
learning_rate = 0.0005
gamma         = 0.98
buffer_limit  = 50000
batch_size    = 32
```

```python
class ReplayBuffer():
    def __init__(self):
        self.buffer = collections.deque(maxlen=buffer_limit)

    def put(self, transition):
        self.buffer.append(transition)

    def sample(self, n):
        mini_batch = random.sample(self.buffer, n)
        s_lst, a_lst, r_lst, s_prime_lst, done_mask_lst = [], [], [], [], []

        for transition in mini_batch:
            s, a, r, s_prime, done_mask = transition
            s_lst.append(s)
            a_lst.append([a])
            r_lst.append([r])
            s_prime_lst.append(s_prime)
            done_mask_lst.append([done_mask])

        return torch.tensor(s_lst, dtype=torch.float), torch.tensor(a_lst), \
               torch.tensor(r_lst), torch.tensor(s_prime_lst, dtype=torch.float), \
               torch.tensor(done_mask_lst)

    def size(self):
        return len(self.buffer)
```

Import Libraries and Hyper-Parameters

<Import Libraries>

- **Torch**: 학습 업데이트에 사용되는 여러 수학적 계산 툴(편미분)을 지원하는 파이썬 라이브러리
- **Gym**: 오픈 AI에서 제공하는 강화 학습 환경 구축을 위한 라이브러리

< Hyper-parameters >

- 학습에 사용되는 Hyper-Parameters를 정의
  - Learning rate (learning_rate)
  - Discount factor (gamma)
  - Size of Reply Buffer (buffer_limit)
  - Size of batch (batch_size)

기초 강화학습 구현

```python
import gym
import collections
import random
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

#Hyperparameters
learning_rate = 0.0005
gamma         = 0.98
buffer_limit  = 50000
batch_size    = 32

class ReplayBuffer():
    def __init__(self):
        self.buffer = collections.deque(maxlen=buffer_limit)

    def put(self, transition):
        self.buffer.append(transition)

    def sample(self, n):
        mini_batch = random.sample(self.buffer, n)
        s_lst, a_lst, r_lst, s_prime_lst, done_mask_lst = [], [], [], [], []

        for transition in mini_batch:
            s, a, r, s_prime, done_mask = transition
            s_lst.append(s)
            a_lst.append([a])
            r_lst.append([r])
            s_prime_lst.append(s_prime)
            done_mask_lst.append([done_mask])

        return torch.tensor(s_lst, dtype=torch.float), torch.tensor(a_lst), \
               torch.tensor(r_lst), torch.tensor(s_prime_lst, dtype=torch.float), \
               torch.tensor(done_mask_lst)

    def size(self):
        return len(self.buffer)
```

Experience Replay

<Reply Buffer Class 정의>

- collections.deque()를 통해 buffer가 꽉 찬 상태에서 추가 데이터가 입력되면 자동으로 오래된 데이터가 buffer에서 빠져나옴
- **def put()**: 학습을 통해 나온 데이터(transition)를 buffer에 넣는 함수
- **def sample()**: 현재 상태/현재 행동/현재 보상/다음 상태/에피소드 종료 여부 끼리 list up하여 torch.tensor형태로 반환

기초 강화학습 구현

```python
class Qnet(nn.Module):
    def __init__(self):
        super(Qnet, self).__init__()
        self.fc1 = nn.Linear(4, 128)
        self.fc2 = nn.Linear(128, 128)
        self.fc3 = nn.Linear(128, 2)

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        return self.fc3(x)

    def sample_action(self, obs, epsilon):
        out = self.forward(obs)
        coin = random.random()
        if coin < epsilon:
            return random.randint(0,1)
        else :
            return out.argmax().item()
```

```python
def train(q, q_target, memory, optimizer):
    for i in range(10):
        s,a,r,s_prime,done_mask = memory.sample(batch_size)

        q_out = q(s)
        q_a = q_out.gather(1,a)
        max_q_prime = q_target(s_prime).max(1)[0].unsqueeze(1)
        target = r + gamma * max_q_prime * done_mask
        loss = F.smooth_l1_loss(q_a, target)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

```python
env = gym.make('CartPole-v1')
q = Qnet()
q_target = Qnet()
q_target.load_state_dict(q.state_dict())
memory = ReplayBuffer()
```

신경망 선언 및 학습 함수 정의

<Q-Network 정의>

- 128개의 노드로 이루어진 2개의 히든 레이어 정의
- CartPole 환경의 특성으로 인하여 state dimension = 4, action dimension = 2로 설정
- **def sample_action()**: 에이전트의 탐험(exploration)을 보장하기 위해 ε –greedy 학습법 사용

<학습 함수 정의>

- 한 번 업데이트 할 때 batch_size의 10배의 데이터를 추출하여 사용
- max_q_prime을 계산할 때, Q-Network가 아닌 Target Network 호출
- Buffer에서 데이터를 추출하고 그것들을 통해 손실함수 계산

```python
def main():
    env = gym.make('CartPole-v1')
    q = Qnet()
    q_target = Qnet()
    q_target.load_state_dict(q.state_dict())
    memory = ReplayBuffer()

    print_interval = 20
    score = 0.0
    optimizer = optim.Adam(q.parameters(), lr=learning_rate)

    for n_epi in range(10000):
        epsilon = max(0.01, 0.08 - 0.01*(n_epi/200)) #Linear annealing from 8% to 1%
        s, _ = env.reset()
        done = False

        while not done:
            a = q.sample_action(torch.from_numpy(s).float(), epsilon)
            s_prime, r, done, truncated, info = env.step(a)
            done_mask = 0.0 if done else 1.0
            memory.put((s,a,r/100.0,s_prime, done_mask))
            s = s_prime

            score += r
            if done:
                break

        if memory.size()>2000:
            train(q, q_target, memory, optimizer)

        if n_epi%print_interval==0 and n_epi!=0:
            q_target.load_state_dict(q.state_dict())
            print("n_episode :{}, score : {:.1f}, n_buffer : {}, eps : {:.1f}%".format(
                                                n_epi, score/print_interval, memory.size(), epsilon*100))
            score = 0.0
    env.close()
```

환경 설정 및 Main 함수 정의

<환경 설정>

- env에 Open AI의 CartPole-v1 환경을 호출
- Q-Network, Target-Network, 그리고 Reply Buffer 정의
- Optimizer는 Adam으로 설정

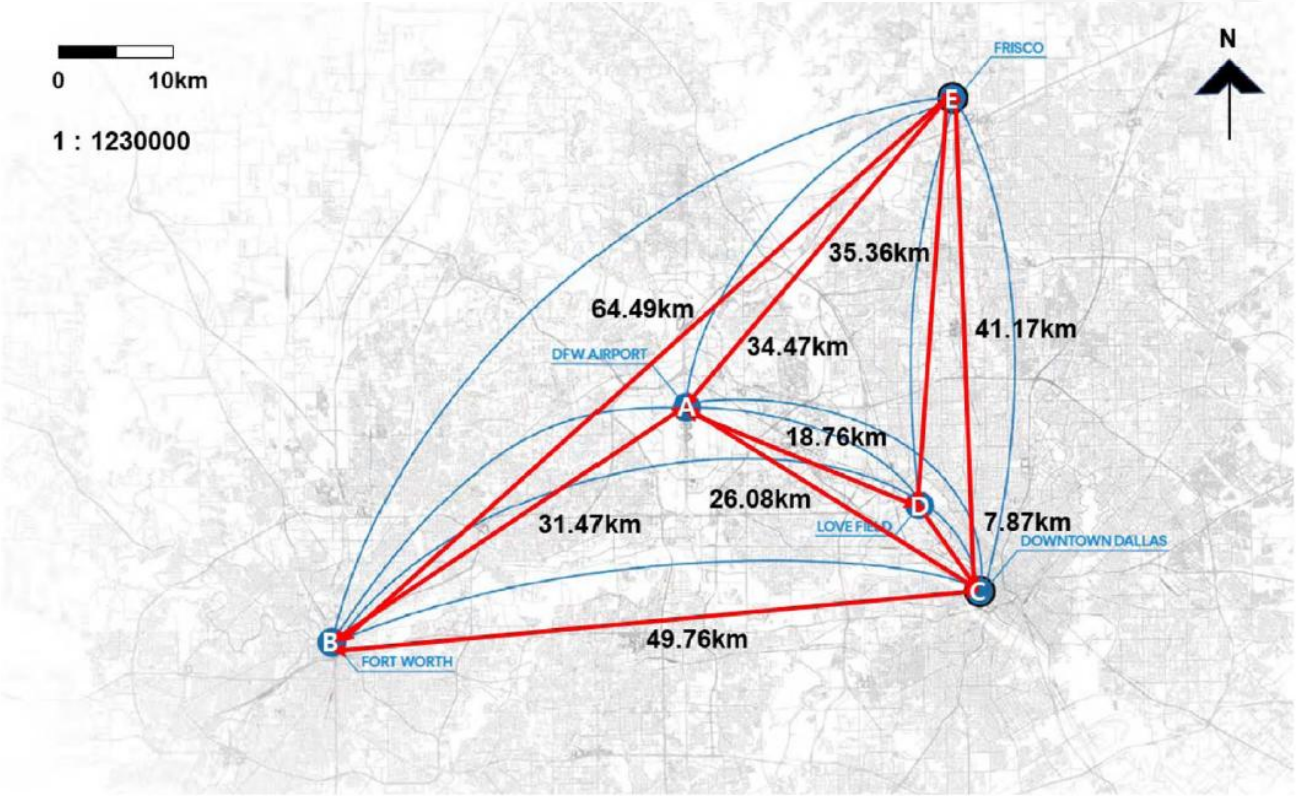
< Main 함수 >

- 학습 에피소드는 10,000으로 설정
- 탐험비율은 8%에 시작하여 1%까지 200 에피소드 마다 1%씩 선형적으로 감소
- 에이전트가 행동을 취하고 그에 따라 상태가 변화하면 해당 Transitions을 Buffer에 저장
- Buffer에 데이터가 2,000개 이상 쌓이면 train을 호출하여 학습을 시작
- 20에피소드 마다 Target Network를 복사해줌

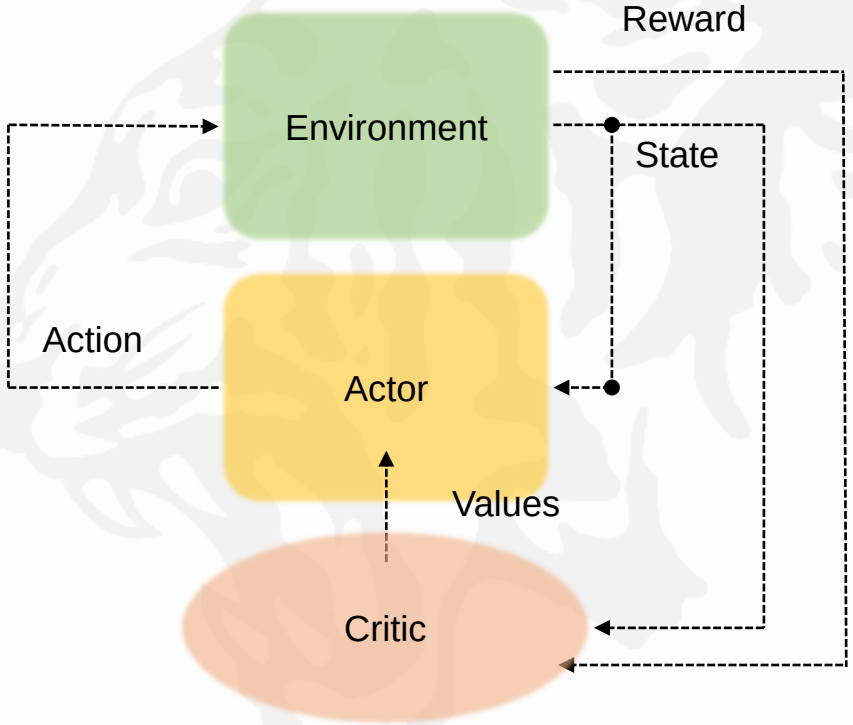
다중 강화학습 기반 무인 이동체 제어

➢Overview

➢Realistic Environment (Vertiport Map)



- Markov Decision Process (MDP) Components: $<S, A, R, T, \gamma>$
  - S: Set of states
  - A: Set of actions
  - R: Reward function
  - T: Transition function
  - γ: Discount factor



다중 강화학습 기반 무인 이동체 제어

- State $\mathcal{S}_{\mathrm{m}} = \{\mathcal{S}_{\mathrm{m}}^{l}, \mathcal{S}_{\mathrm{m}}^{s}, \mathcal{S}_{\mathrm{m}}^{e}\}$
  - Set of Passengers : $\mathcal{U} \triangleq \{\mathrm{u}_1, \dots, \mathrm{u}_{\mathrm{n}}, \dots, \mathrm{u}_{\mathrm{N}}\}$
  - Set of UAM Agents : $\mathcal{B} \triangleq \{\mathrm{b}_1, \dots, \mathrm{b}_{\mathrm{m}}, \dots, \mathrm{b}_{\mathrm{M}}\}$
  - Set of Vertiports : $\mathcal{V} \triangleq \{\mathrm{v}_1, \dots, \mathrm{v}_{\mathrm{k}}, \dots, \mathrm{v}_{\mathrm{K}}\}$

- a. Location Information
  - $\mathcal{S}_{\mathrm{m}}^{\mathrm{l}} = \{\mathrm{p}(\mathrm{b}_{\mathrm{m}}), \mathrm{d}(\mathrm{b}_{\mathrm{m}}, b_{\mathrm{m}'}), d(b_m, v_k)\}$

- b. Service Information
  - $\mathcal{S}_{\mathrm{m}}^{\mathrm{s}} = \{\cup_{n=1}^{N}\{\chi_{mn}\}, \eta_m, \Psi_m, \tau_m\}$

- c. Energy Information
  - $\mathcal{S}_{\mathrm{m}}^{\mathrm{e}} = \{e_m\}$

(a) Round-Trip Traveling

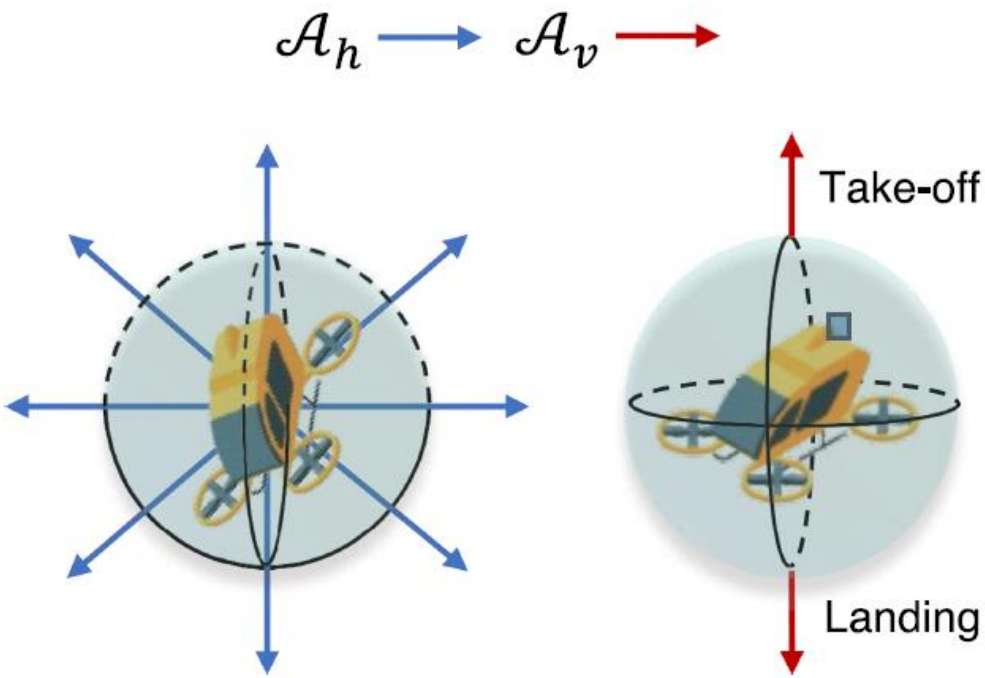
$$P_p = \underbrace{P_i\left(\sqrt{1+\frac{v^4}{4v_0^4}} - \frac{v^2}{2v_0^2}\right)^{0.5}}_{induced} + \underbrace{P_0\left(1+\frac{3v^2}{U_{tip}^2}\right)}_{bladeprofile} + \underbrace{\frac{1}{2}d_0\rho s A v^3}_{parasite},$$

(b) Hovering / Ascending / Descending

$$P_h = \underbrace{\frac{C_d}{8}\rho s A \Omega^3 R^3}_{blade\ profile,\ P_o} + \underbrace{(1+k)\frac{W^{\frac{3}{2}}}{\sqrt{2\rho A}}}_{induced,\ P_i},$$

다중 강화학습 기반 무인 이동체 제어

- Action $\mathcal{A} = \{\mathcal{A}_{\mathrm{p}}, \mathcal{A}_{\mathrm{v}}\}$
  - $\mathcal{A}_{\mathrm{h}} = \{\mathrm{x_{pos}} \pm (\mathrm{v} \times \mathrm{t}), \mathrm{y_{pos}} \pm (\mathrm{v} \times \mathrm{t})\}$
  - $\mathcal{A}_{\mathrm{v}} = = \{\mathrm{hovering}, \mathrm{takeoff}, \mathrm{landing}\}$



(a) Top View. (b) Forward View.

다중 강화학습 기반 무인 이동체 제어

➢ Reward : $R^m(t) = \rho_s \times R_c(t) \times (1 + R_i^m(t))$

- Common Reward
- $R_c(t) = \sum_{j=1}^{N} \frac{1}{1+\sum_0^t W^j(t)}$

- Individual Reward
- $R_i^m(t) = R_s^m(t) \times R_e^m(t)$
- $R_e^m(t) = e_m(t) - (P_h \times \mathcal{A}_h + P_p \times \mathcal{A}_p)$

Algorithm 1: Policy Training and Inference in CTDE.

Initialize parameters of *actor* networks and *centralized critic* network which are denoted as $[\boldsymbol{\theta}_1, \cdots, \boldsymbol{\theta}_j, \cdots, \boldsymbol{\theta}_J]$ and $\boldsymbol{\phi}$;
Initialize replay buffer $\mathcal{D} = \{\}$ and mini-batch $\mathcal{B} = \{\}$;
for *Epoch = 1, MaxEpoch* **do**
  ▷ **Initialize Air Transportation Environments**, set s_0;
  for *time step = 1, T* **do**
    for *each j-th UAM* **do**
      ▷ Select the action a_j based on its policy $\pi_{\boldsymbol{\theta}_j}(a_j^t \mid o_j^t)$ at time step t;
    end
    ▷ $s^t \rightarrow s^{t+1}$, $\mathbf{o}^t \rightarrow \mathbf{o}^{t+1}$ with the reward $\mathbf{r}^t$;
    ▷ Set $\xi = \{s^t, \mathbf{o}^t, \mathbf{a}^t, \mathbf{r}^t, s^{t+1}, \mathbf{o}^{t+1}\}$;
    ▷ Update $\mathcal{D}$: $Enqueue(\mathcal{D}, \xi)$;
  end
  if $\mathcal{D}$ ***is full enough to train*** **then**
    for *each j-th UAM* **do**
      ▷ Get V_ϕ by sampling mini-batch $\mathcal{B}$ from $\mathcal{D}$;
      ▷ Update ϕ by **gradient descent** to loss function of the *centralized critic* network: $\nabla_\phi \mathcal{L}(\phi)$;
      ▷ Update $\boldsymbol{\theta}_j$ by **gradient ascent** to objective function of j-th *actor* network: $\nabla_{\boldsymbol{\theta}_j} J(\boldsymbol{\theta}_j)$;
    end
  end
end
for *Episode = 1, MaxEpisode* **do**
  ▷ **Initialize Air Transportation Environments**, set s_0;
  for *time step = 1, T* **do**
    for *each j-th UAM* **do**
      ▷ Select the action a_j based on its policy $\pi_{\boldsymbol{\theta}_j}(a_j^t \mid o_j^t)$ at time step t;
    end
    ▷ $s^t \rightarrow s^{t+1}$, $\mathbf{o}^t \rightarrow \mathbf{o}^{t+1}$;
  end
end

다중 강화학습 기반 무인 이동체 제어

➢UAM Model

CityAirbus – Airbus Helicopters

Another project in development is the *CityAirbus*, by *AIRBUS* (Figure 2.5). This project is an electrically powered EVTOL aircraft demonstrator, so it aims to settle the base for the future UAT. To avoid ground traffic congestion, *AIRBUS* propose an all electrical four seat UAT. One pilot will be required to transport the passengers. The 8 electric ducted fan fed by 8 electrical motors of 100 kW each one will make the *CityAirbus* fly. The four batteries with a capacity of 110 kW for each battery makes a total amount of 110 kWh. This makes the maximum autonomy of the UAT to be 15 minutes, with a cruise speed of 120 km/h.

함부르크 응용 과학 대학,
Evaluation of the Hybrid-Electric Aircraft Project Airbus E-Fan X

Parameters	Value
Cruise Speed	120 km/h
Flight time	15 min
Battery	110 kWh
Passengers	4
Take-off weight	2,200 kg
Weight	1,850 kg
Charging time	30 - 60 min

➢UAM Model

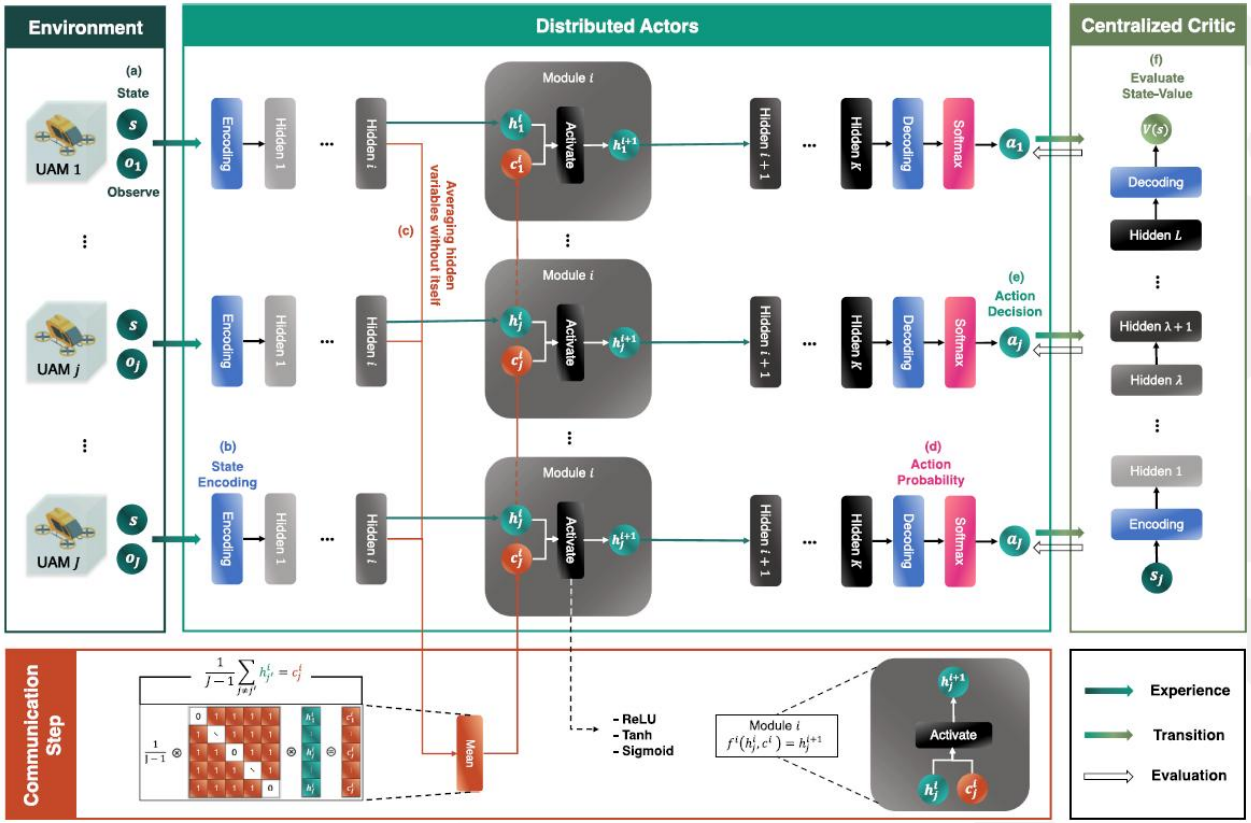
$$E_{hc}(t) = (P_h + P_c) \cdot t$$

Parameters	Value
Cruise Speed	120 km/h
Flight time	15 min
Battery	110 kWh
Passengers	4
Take-off weight	2,312 kg
Weight	1,850 kg
Charging time	30 - 60 min

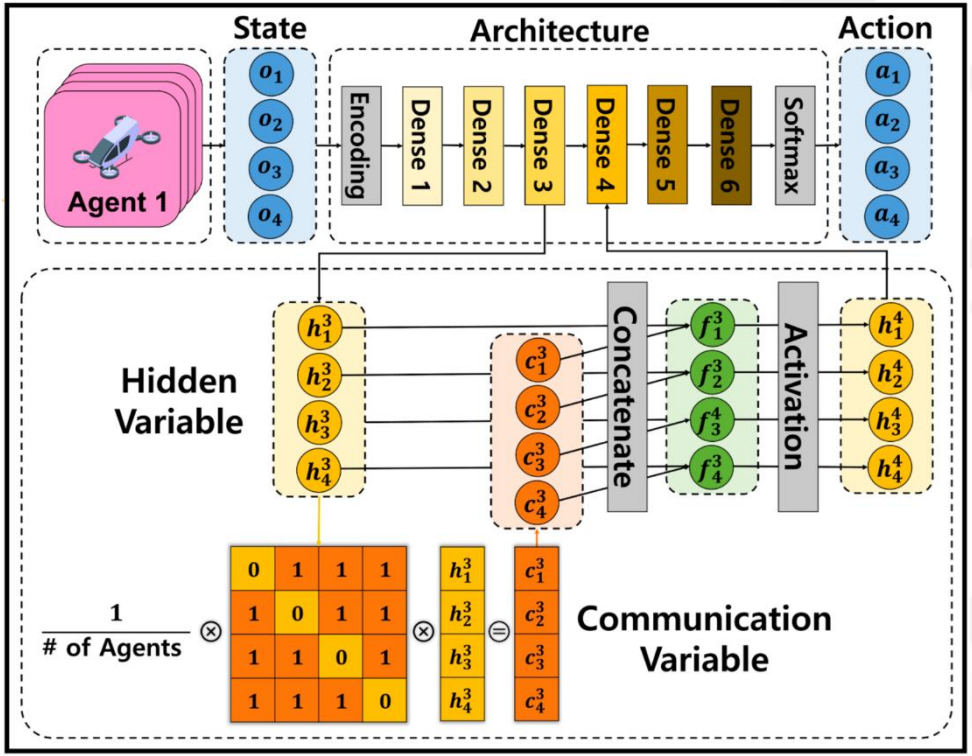
Notation	Value
Aircraft weight including battery and propellers, W	1850 kg
Rotor radius, R	1.4 m
Rotor disc area, $A = \pi R^2$	6.16 m^2
Number of blades, b	8
Rotor solidity, $s = \frac{0.0157b}{\pi R}$	0.03
Blade angular velocity, Ω	100 rad/s
Tip speed of the rotor blade, $U_{tip} = \Omega R^2$	196
Fuselage drag ratio, $d_0 = \frac{0.0151}{sA}$	0.086
Air density, ρ	1.225 kg/m^3
Mean rotor-induced velocity in hovering, $v_0 = \sqrt{\frac{W}{s\rho A}}$	90.4
Profile drag coefficient, δ	0.012
Incremental correction factor to induced power, k	0.1

Parameters	Meaning
Ph	Hovering Power
Pc	Wireless Communication Power
Eh	Hovering Energy
FM	“figure of merit” of the aircraft
s	Flying speed
l	Flying distance
m	mass
ρ	Density of Air
A_r	Overall Rotor Area
g	Gravity Acceleration

➢Network Architecture



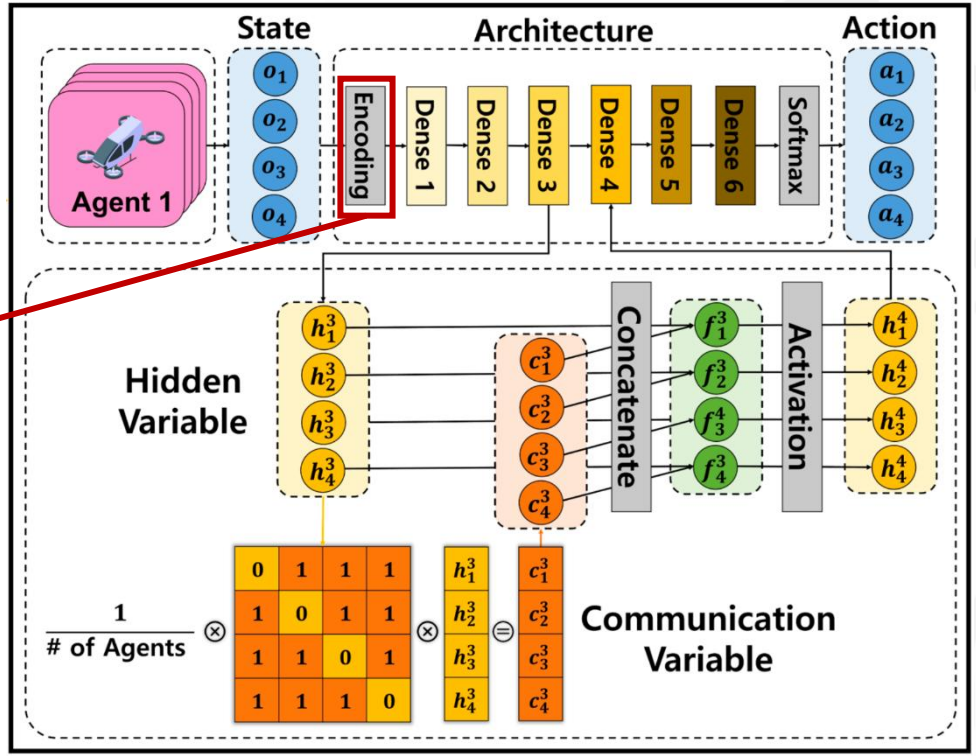
➢CommNet (When Backpropagation)



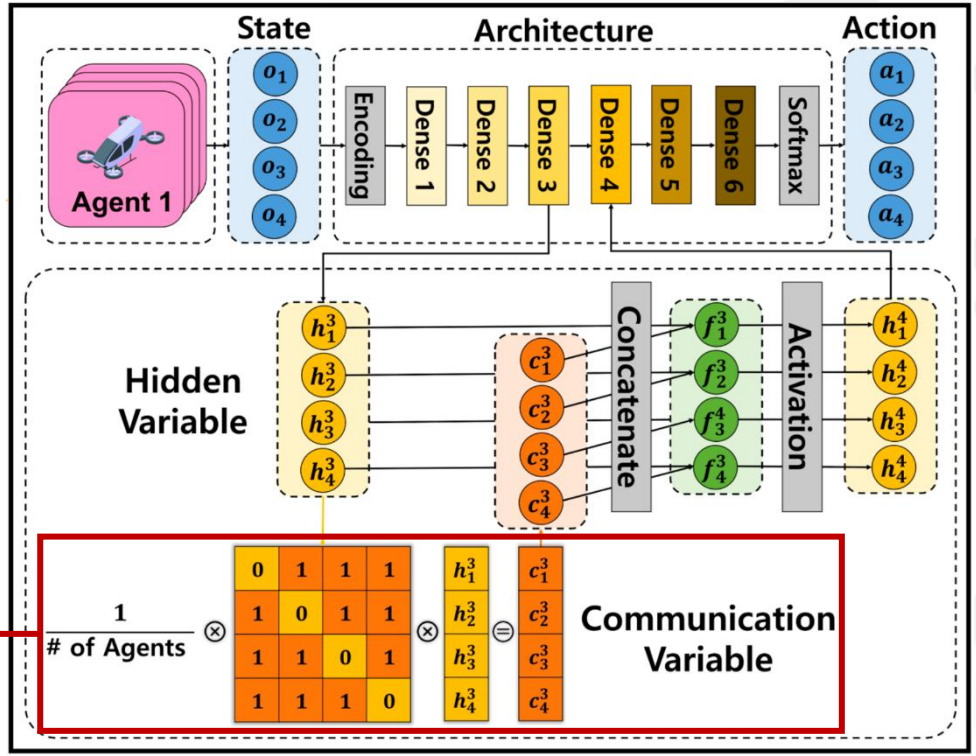
➢CommNet (When Backpropagation)

(1) Encoding

$$h_m^1 = \text{Encoder}(o_m)$$



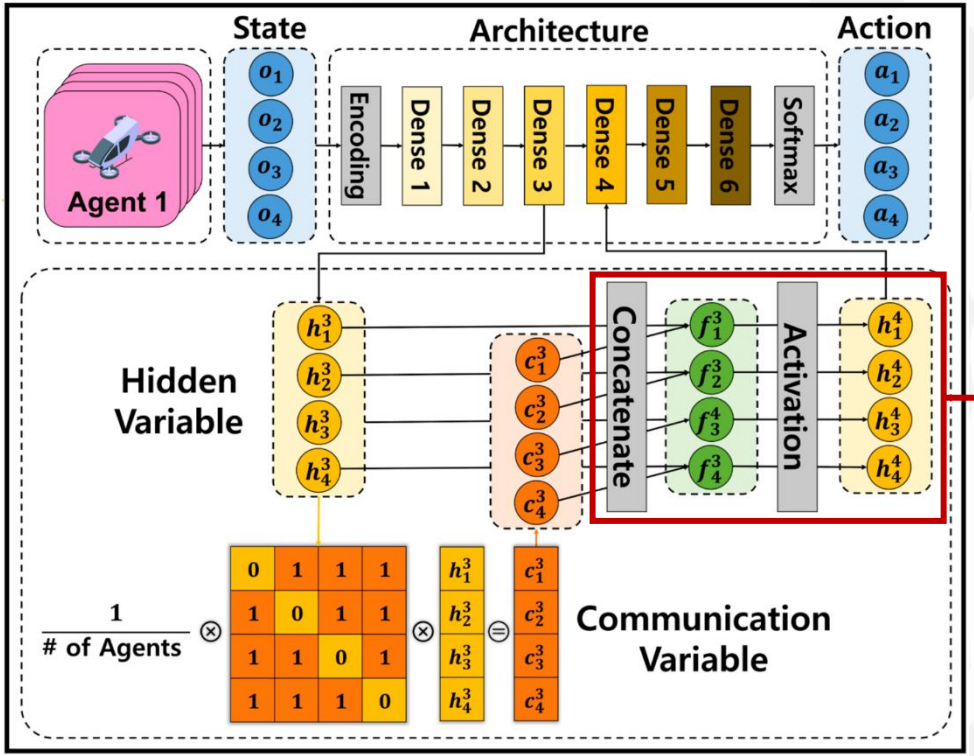
➢CommNet (When Backpropagation)



(2) Communication Step

$$c_m^l = \frac{1}{M-1}\sum_{m \neq M} h_m^l$$

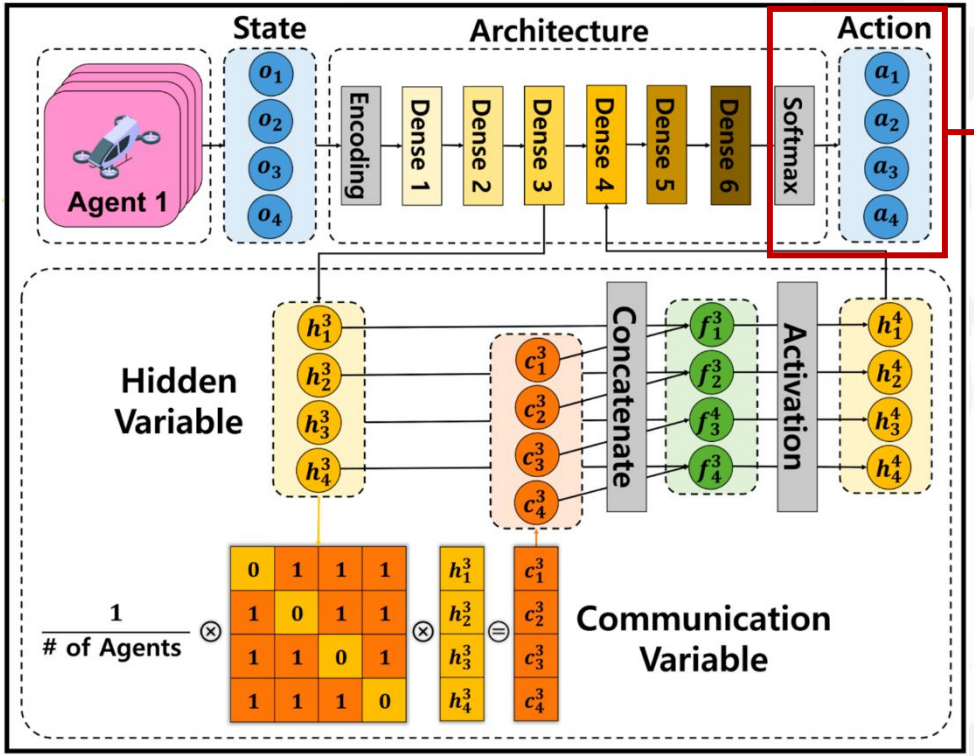
➢CommNet (When Backpropagation)



(3) Generate Next Hidden Variable

$$h_m^{l+1} = \text{Activ}(f_m^l(\text{Concat}(h_m^l, c_m^l)))$$

➢CommNet (When Backpropagation)

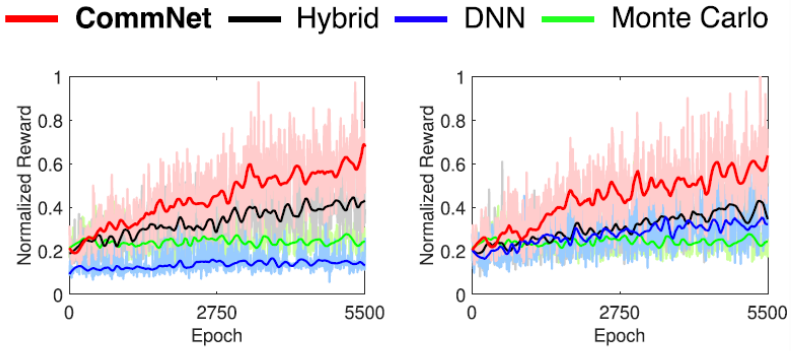


(4) Action Decision

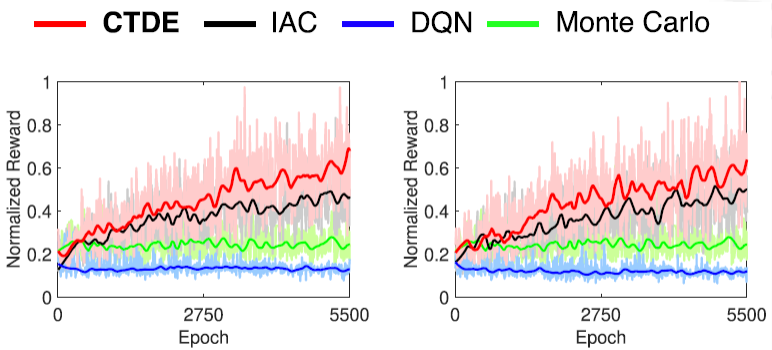
$$a_m = \text{Softmax}(\text{Concat}([h_1^L, \cdots, h_M^L]))$$

강화학습 기반 제어기 성능평가

➢Performance Evaluation (Reward)



(a) POMDP. (b) FOMDP.



(a) POMDP. (b) FOMDP.

REWARD COMPARISON TO INVESTIGATE THE EFFECT OF COMMNET

1 : Reward Convergence in POMDP and FOMDP

Algorithm	POMDP	FOMDP
CommNet (Proposed)	0.647	0.602
Hybrid	0.438	0.456
DNN	0.148	0.337
Monte Carlo	0.265	0.265

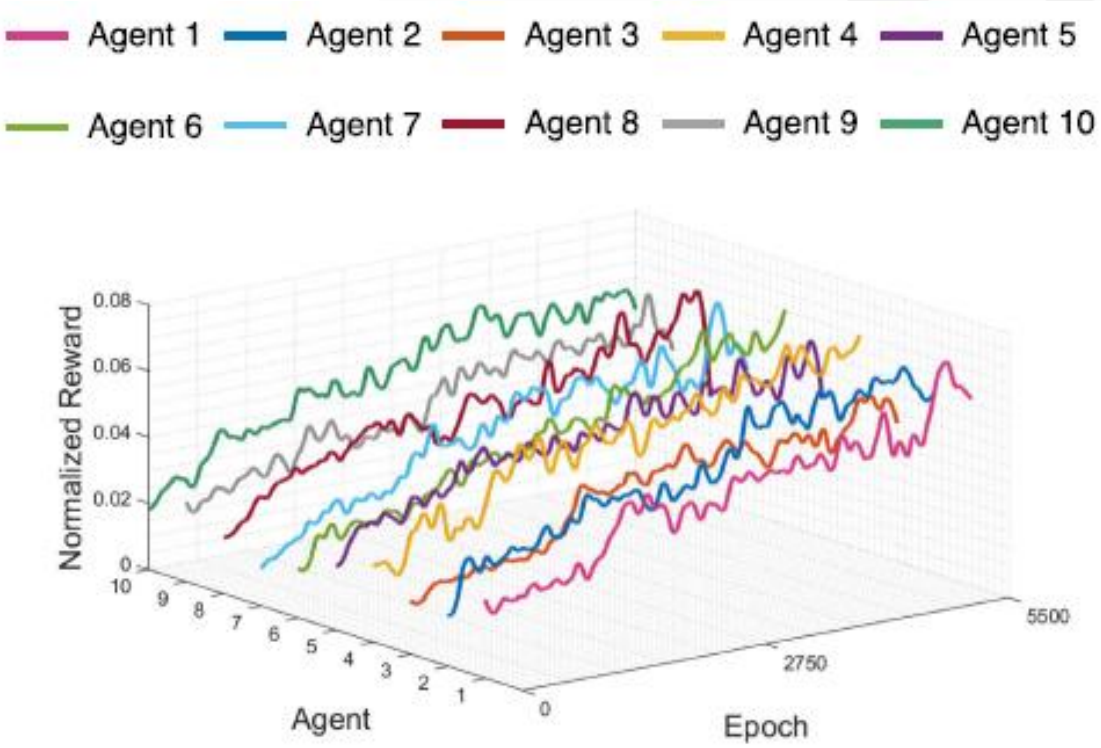
REWARD COMPARISON TO INVESTIGATE THE EFFECT OF CTDE

2 : Reward Convergence in POMDP and FOMDP

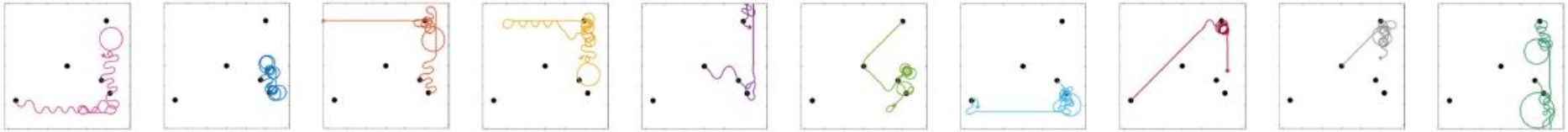
Algorithm	POMDP	FOMDP
CTDE (Proposed)	0.647	0.602
IAC	0.440	0.481
DQN	0.131	0.111
Monte Carlo	0.265	0.265

강화학습 기반 제어기 성능평가

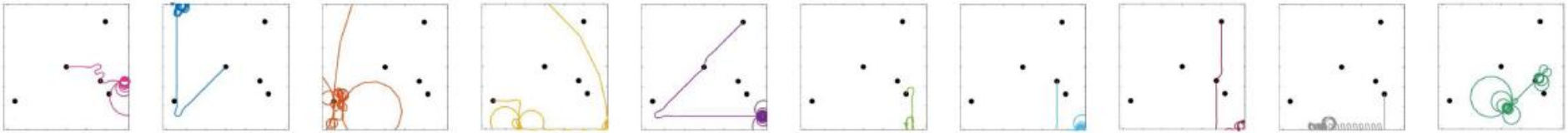
➢ Performance Evaluation (Reward)



➢Performance Evaluation (Trajectories)

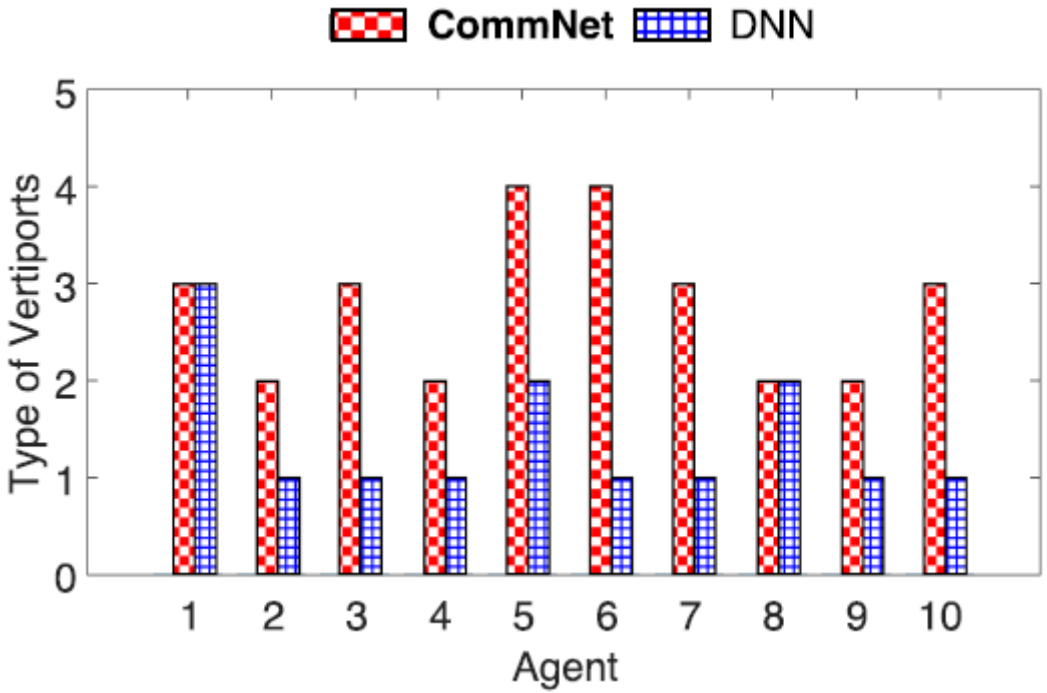


(a) Agent 1. (b) Agent 2. (c) Agent 3. (d) Agent 4. (e) Agent 5. (f) Agent 6. (g) Agent 7. (h) Agent 8. (i) Agent 9. (j) Agent 10.

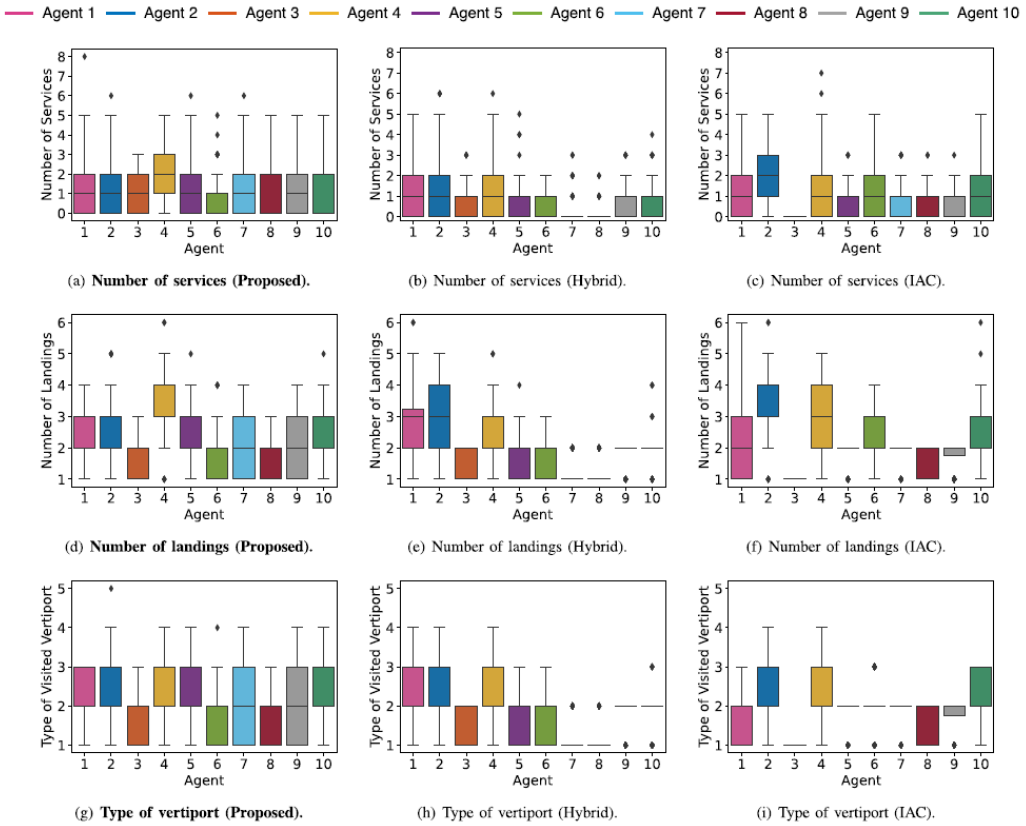


(a) Agent 1. (b) Agent 2. (c) Agent 3. (d) Agent 4. (e) Agent 5. (f) Agent 6. (g) Agent 7. (h) Agent 8. (i) Agent 9. (j) Agent 10.

➢ Performance Evaluation (Type of Vertiports)



➢Performance Evaluation (Total Air Transportation Service Quality of all UAMs)



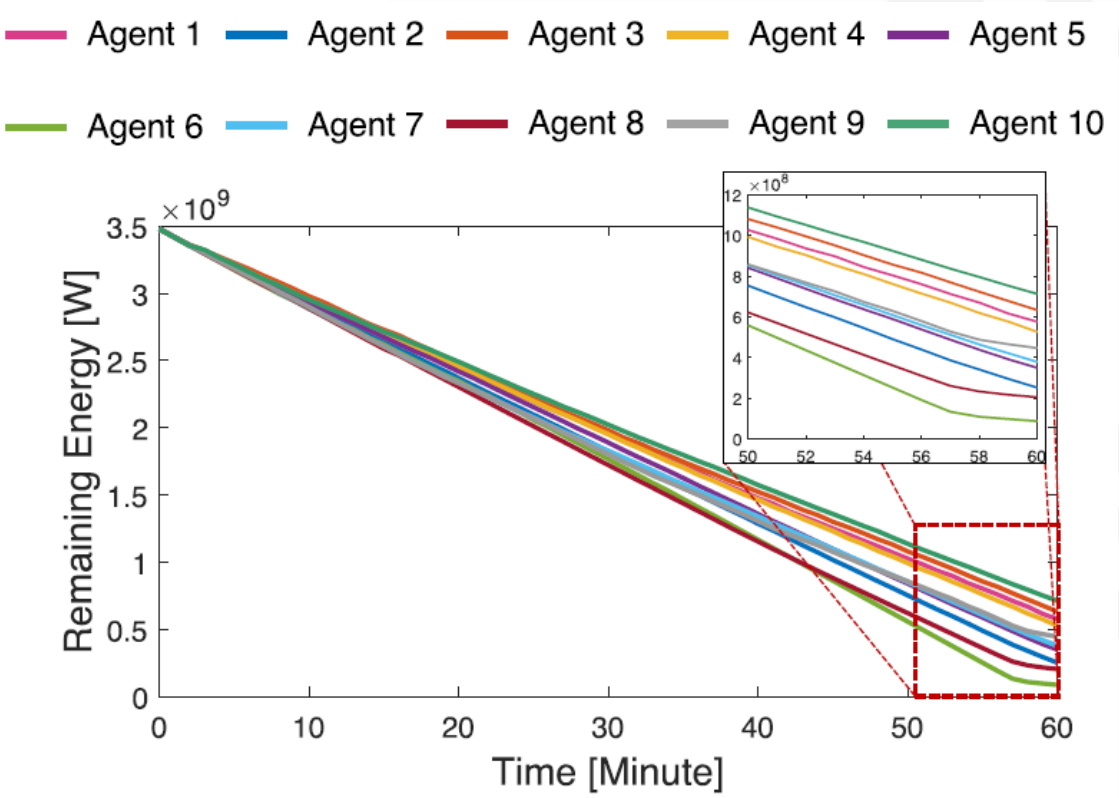
(a) **Number of services (Proposed).** (b) Number of services (Hybrid). (c) Number of services (IAC).

(d) **Number of landings (Proposed).** (e) Number of landings (Hybrid). (f) Number of landings (IAC).

(g) **Type of vertiport (Proposed).** (h) Type of vertiport (Hybrid). (i) Type of vertiport (IAC).

강화학습 기반 제어기 성능평가

➢ Performance Evaluation (Residual Energy)



무인 이동체 제어 구현 (Main Function)


```python
from argument import get_args, get_env_info
from environmentComm import Environment
from memory import ReplayBuffer, get_experience
import torch
import copy
import random
import numpy as np
import pandas as pd
▶ Launch TensorBoard Session
from torch.utils.tensorboard import SummaryWriter
from datetime import datetime
from agent import Agent, Critic_V
import json

DICT = {
    'Total':[],
    'Agent1':[],
    'Agent2':[],
    'Agent3':[],
    'Agent4':[],
    'Agent5':[],
    'Agent6':[],
    'Agent7':[],
    'Agent8':[],
    'Agent9':[],
    'Agent10':[],
}

env = Environment()
args = get_args()
args = get_env_info(args, env)

random.seed(args.seed)
np.random.seed(args.seed)
torch.manual_seed(args.seed)

def detach_cuda(Agents):
    for agent in Agents:
        agent.actor = agent.actor.cpu()
        agent.actor_target = agent.actor_target.cpu()

def make_cuda(Agents):
    for agent in Agents:
        agent.actor = agent.actor.cuda()
        agent.actor_target = agent.actor_target.cuda()
```

```python
Agents = [Agent(args = args, id = i) for i in range(args.n_agents)]
if args.mode == 'IAC': Critics = [Critic_V(args=args) for _ in range(args.n_agents)]
else : Central_V = Critic_V(args)
replay = ReplayBuffer(args)
reward_max = -10000
today = datetime.today().strftime("%m%d%H%M%S")
info  = f'{args.mode}_Comm{args.n_Comm_agents}_DNN{args.n_DQN_agents}_lr{args.actor_lr}'
writer = SummaryWriter(f'runs/{today}+{info}')
x_COMM, y_COMM, x_DNN, y_DNN = None, None, None, None
for epoch in range(args.train_epoch+1):
    env.reset()
    o       = env.o # n_agents, obs_dim
    o_prime = env.o_prime
    ava     = env.get_available_action()
    TB_RWD  = np.zeros(args.n_agents)
    TB_EN   = np.zeros(args.n_agents)
    # replay.buffer.clear()
    takeoff = None
    service = np.zeros([args.episode_limit+1, args.n_agents+1])
    energy  = np.zeros([args.episode_limit+1, args.n_agents])
    # replay.initialize()
    for t in range(args.episode_limit+1):
        o = o_prime.copy()
        actions, u_onehot = [], np.zeros((args.n_agents,args.n_actions))
        for i in range(args.n_agents):
            coin = np.random.rand()
            if args.mode != 'Random':
                if coin >= args.epsilon:
                    action = Agents[i].select_action(id=i, o=o, ava=ava[i]).squeeze().data.cpu().numpy()
                else:
                    while(True):
                        action = np.random.randint(low=0,high=args.n_actions)
                        if ava[i][action] == 1: break
            else:
                while(True):
                    coin = np.random.rand()
                    action = np.random.randint(low=0,high=args.n_actions)
                    if ava[i][action] == 1: break
            actions.append(action)
        actions = np.array(actions)
        o, rewards, o_prime, ava, takeoff, x_COMM, y_COMM, x_DNN, y_DNN = env.step(actions, t, epoch, today, info)
        # experience = get_experience(o, actions, rewards.sum(), o_prime)
        experience = get_experience(o, actions, rewards, o_prime)
        replay.push(experience)
        TB_RWD += rewards
        _, TB_EN = env.utility.get_utils_info()
```

무인 이동체 제어 구현 (Main Function)


```python
        for i in range(args.n_agents+1) : service[t][i] = env.utility.SERVICED[i]
        for i in range(args.n_agents)   :  energy[t][i] = TB_EN[i]

    # [1] Sampling Batch Experiences
    args.epsilon -= args.anneal_epsilon
    args.epsilon  = max(args.epsilon, args.min_epsilon)

    if args.mode != 'Random':
        if replay.size() > args.train_size:
            samples = replay.sample()
            O = samples['O']
            ACTIONS = samples['A']
            REWARDS = samples['REWARDS']
            O_PRIME = samples['O_PRIME']
            ACTOR_LOSS  = []
            # CRITIC_LOSS = []

            for i in range(args.n_agents):
                # actor_loss = Agents[i].train(O,ACTIONS[:,i],REWARDS,O_PRIME,Central_V)
                if args.mode == 'IAC': actor_loss = Agents[i].train(O,ACTIONS[:,i],REWARDS[:,i],O_PRIME,Critics[i])
                else:                  actor_loss = Agents[i].train(O,ACTIONS[:,i],REWARDS[:,i],O_PRIME,Central_V)
                ACTOR_LOSS.append(actor_loss)
                # CRITIC_LOSS.append(critic_loss)
            ACTOR_LOSS  = np.array(ACTOR_LOSS)
            # CRITIC_LOSS = np.array(CRITIC_LOSS)

            if args.mode == 'DQN' or args.mode == 'DDPG':
                if epoch != 0 and epoch % args.target_update_cycle == 0:
                    detach_cuda(Agents)
                    for agent in Agents:
                        agent.actor_target.load_state_dict(agent.actor.state_dict())
                    make_cuda(Agents)

            # writer.add_scalars(f'loss/critic', {
            #     'critic': CRITIC_LOSS.sum() / args.n_agents
            # }, epoch)

            writer.add_scalars(f'loss/actor', {
                'total' : ACTOR_LOSS.sum(),
                'agent1': ACTOR_LOSS[0],
                'agent2': ACTOR_LOSS[1],
                'agent3': ACTOR_LOSS[2],
                'agent4': ACTOR_LOSS[3],
                'agent5': ACTOR_LOSS[4],
                'agent6': ACTOR_LOSS[5],
                'agent7': ACTOR_LOSS[6],
                'agent8': ACTOR_LOSS[7],
                'agent9': ACTOR_LOSS[8],
```

```python
    for i in range(args.n_agents):
        total = 0
        for j in range(5):
            if takeoff[i][j] > 0:
                total += 1
        writer.add_scalars(f'takeoff/agent{i}',{
                'numVertiport': total,
                'vertiport1': takeoff[i][0],
                'vertiport2': takeoff[i][1],
                'vertiport3': takeoff[i][2],
                'vertiport4': takeoff[i][3],
                'vertiport5': takeoff[i][4]
            }, epoch)

    reward_max = max(reward_max, TB_RWD.sum())
    writer.add_scalars(f'Reward/reward', {
            'total' : TB_RWD.sum(),
            'agent1': TB_RWD[0],
            'agent2': TB_RWD[1],
            'agent3': TB_RWD[2],
            'agent4': TB_RWD[3],
            'agent5': TB_RWD[4],
            'agent6': TB_RWD[5],
            'agent7': TB_RWD[6],
            'agent8': TB_RWD[7],
            'agent9': TB_RWD[8],
            'agent10': TB_RWD[9],
        }, epoch)

    DICT['Total'].append(TB_RWD.sum())
    DICT['Agent1'].append(TB_RWD[0])
    DICT['Agent2'].append(TB_RWD[1])
    DICT['Agent3'].append(TB_RWD[2])
    DICT['Agent4'].append(TB_RWD[3])
    DICT['Agent5'].append(TB_RWD[4])
    DICT['Agent6'].append(TB_RWD[5])
    DICT['Agent7'].append(TB_RWD[6])
    DICT['Agent8'].append(TB_RWD[7])
    DICT['Agent9'].append(TB_RWD[8])
    DICT['Agent10'].append(TB_RWD[9])
```

무인 이동체 제어 구현 (Agent Function)


```python
import numpy as np
import torch
from torch.distributions import Categorical
from policy import DQNActor, CommNetActor, Critic
from copy import deepcopy
import torch.nn.functional as f
import torch.nn as nn
# temperature = 5

class Agent:
    def __init__(self, args, id = None):
        self.id        = id
        self.args      = args
        self.n_actions = args.n_actions
        # self.state_shape = args.state_dim
        self.obs_shape = args.obs_dim
        if id < args.n_Comm_agents : self.actor = CommNetActor(args)
        else                       : self.actor = DQNActor(args)
        self.actor_target = deepcopy(self.actor)
        self.actor.cuda()
        self.actor_target.cuda()
        self.actor_optimizer = torch.optim.Adadelta(self.actor.parameters(), lr=self.args.actor_lr)

    def o_preprocessing(self, o):
        if self.id < self.args.n_Comm_agents:
            agent_obs   = np.copy(o)
            IDset       = np.arange(self.args.n_agents)
            logic_IDset = IDset == self.id
            logic_IDset = np.where(logic_IDset == 0)[0]
            logic_IDset = np.hstack([[self.id],logic_IDset])
            agent_obs   = agent_obs[logic_IDset,:]
            agent_obs   = torch.FloatTensor(agent_obs)
        else :
            agent_obs = torch.FloatTensor(o)[self.id]
        return agent_obs.unsqueeze(0)

    def _O_preprocessing(self, O):
        if self.id < self.args.n_Comm_agents:
            agent_obs   = np.copy(O)
            IDset       = np.arange(self.args.n_agents)
            logic_IDset = IDset == self.id
            logic_IDset = np.where(logic_IDset == 0)[0]
            logic_IDset = np.hstack([[self.id],logic_IDset])
            agent_obs   = agent_obs[:,logic_IDset,:]
            agent_obs   = torch.FloatTensor(agent_obs)
        else:
            agent_obs = torch.FloatTensor(O[:,self.id,:])
        return agent_obs

    def O_preprocessing(self, O):
        agent_obs = torch.FloatTensor(O[:,self.id,:])
        return agent_obs

    def _choose_action_from_softmax(self, action_dist, ava, id):
        ava    = torch.FloatTensor(ava).cuda()
        prob   = f.softmax(action_dist, dim=-1) * ava
        # prob   = prob[0][id]
        action = torch.argmax(prob,-1)
        return action

    def select_action(self, id, o, ava):
        agent_obs = self.o_preprocessing(o)
        action_dist = self.actor(agent_obs.cuda())
        action  = self._choose_action_from_softmax(action_dist, ava, id)
        return action


    def train(self,O,A,RWD,O_PRIME,critic): # ==> Centralized Critic Observes Agentwise Observation Information
        obs       = O
        obs_prime = O_PRIME
        O         = self._O_preprocessing(O)
        O_PRIME   = self._O_preprocessing(O_PRIME)
        O_C       = self.O_preprocessing(obs)
        O_C_PRIME = self.O_preprocessing(obs_prime)
        O         = torch.FloatTensor(O).cuda()
        O_PRIME   = torch.FloatTensor(O_PRIME).cuda()
        O_C       = torch.FloatTensor(O_C).cuda()
        O_C_PRIME = torch.FloatTensor(O_C_PRIME).cuda()
        r         = torch.FloatTensor(RWD).squeeze().cuda().unsqueeze(-1)
        A         = torch.tensor(A.reshape(-1,1),dtype=torch.int64).cuda()
        Q         = self.actor(O).gather(dim=1, index=A)
        if self.args.mode == 'CTDE':
            TD_TARGET = r + self.args.gamma * critic.critic(O_C_PRIME)
            DELTA     = TD_TARGET - critic.critic(O_C)
            loss      = - torch.log(Q) * DELTA.detach() + nn.MSELoss()(critic.critic(O_C), TD_TARGET.detach())

            self.actor_optimizer.zero_grad()
            critic.critic_optimizer.zero_grad()
            loss.mean().backward()

            torch.nn.utils.clip_grad_norm_(self.actor.parameters(), self.args.grad_norm_clip)
            self.actor_optimizer.step()
            self.actor_optimizer.zero_grad()
```

무인 이동체 제어 구현 (Agent Function)


```python
            torch.nn.utils.clip_grad_norm_(critic.critic.parameters(), self.args.grad_norm_clip)
            critic.critic_optimizer.step()
            critic.critic_optimizer.zero_grad()

            output = loss.mean().abs().item()

        elif self.args.mode == 'IAC' or self.args.mode == 'DDPG':
            # critic_optimizer = torch.optim.Adadelta(critic.parameters(), lr=self.args.actor_lr)
            if   self.args.mode == 'IAC'  : TD_TARGET = r + self.args.gamma * critic.critic(O_C_PRIME)
            elif self.args.mode == 'DDPG' : TD_TARGET = r + self.args.gamma * critic.critic(O_C_PRIME)
            DELTA     = TD_TARGET - critic.critic(O_C)
            loss      = - torch.log(Q) * DELTA.detach() + nn.MSELoss()(critic.critic(O_C), TD_TARGET.detach())

            self.actor_optimizer.zero_grad()
            critic.critic_optimizer.zero_grad()
            loss.mean().backward()

            torch.nn.utils.clip_grad_norm_(self.actor.parameters(), self.args.grad_norm_clip)
            self.actor_optimizer.step()
            self.actor_optimizer.zero_grad()

            torch.nn.utils.clip_grad_norm_(critic.critic.parameters(), self.args.grad_norm_clip)
            critic.critic_optimizer.step()
            critic.critic_optimizer.zero_grad()

            output = loss.mean().abs().item()

        elif self.args.mode == 'DQN' or self.args.mode == 'IQL':
            if   self.args.mode == 'DQN' : Q_TARGET = r + self.args.gamma * self.actor_target(O_PRIME).max(-1).values.detach().unsqueeze(-1)
            elif self.args.mode == 'IQL' : Q_TARGET = r + self.args.gamma * self.actor(O_PRIME).max(-1).values.detach().unsqueeze(-1)
            loss = nn.MSELoss()(Q, Q_TARGET.detach())
            self.actor_optimizer.zero_grad()
            loss.backward()
            torch.nn.utils.clip_grad_norm_(self.actor.parameters(), self.args.grad_norm_clip)
            self.actor_optimizer.step()
            self.actor_optimizer.zero_grad()

            output = loss.item()

        return output
```

무인 이동체 제어 구현 (Environment Function)


```python
import numpy as np
import pandas as pd
import math
import os
import matplotlib
matplotlib.use('Agg')
from matplotlib import pyplot as plt
import json
import io
import PIL.Image
from torchvision.transforms import ToTensor

os.chdir(os.path.dirname(__file__)) # 실행 경로를 현재 .py 파일이 위치한 곳(서버의 위치)으로 변경

EP_LEN      = 60
N_VERTIPORT = 5
N_USER      = 200
N_COMMAGENT = 10
N_DQNAGENT  = 0
GRID        = 32000 # [m]
OBSERVABLE  = GRID * 3 # GRID * 1.5 / 3
H_MAX       = 600   # [m] ==> UAM이 horizontal moving하는 altitude에 해당
H_MIN       = 0     # [m] ==> Vertiport의 altitude에 해당
VELOCITY    = 4425  # [m/min]
MTOW        = 4

font = {'size'   :10}
plt.rc('font', **font)   # pass in the font dict as kwargs
from matplotlib import rcParams
rcParams.update({'figure.autolayout': True})

class Vertiport:

    def __init__(self, id = None, x = None, y = None, z = None):

        self.id     = id
        self.x      = x
        self.y      = y
        self.z      = z

class User:

    def __init__(self, id = None, Departure = None, Arrival = None):

        self.id             = id
        self.Arrival        = Arrival    # 도착 Vertiport의 no.
        self.Departure      = Departure  # 출발 Vertiport의 no.
        self.connect        = { 'drone'     : -1,
                                'isOnboard' : 0.
```

```python
class User:

    def __init__(self, id = None, Departure = None, Arrival = None):

        self.id             = id
        self.Arrival        = Arrival    # 도착 Vertiport의 no.
        self.Departure      = Departure  # 출발 Vertiport의 no.
        self.connect        = { 'drone'     : -1,
                                'isOnboard' : 0,
                                'isArrive'  : 0 }

    def is_support(self):

        return self.connect['drone'], self.connect['isOnboard'], self.connect['isArrive']

class Drone:

    def __init__(self, id = None, x = None, y = None, z = None, vertiport = None):

        self.id             = id
        self.x              = x
        self.y              = y
        self.z              = z
        self.vertiport      = vertiport
        self.onboarding     = 0
        self.counter        = 4
        self.takeoff        = 0
        self.MTOW           = MTOW
        self.numUser        = 0
        self.distance       = 0
        self.isworking      = 1
        self.seat           = np.full([self.MTOW], -1, dtype=int)
        self.batteryRemain  = 4 * 5.4 * 100000000 # [J]
        self.maxbattery     = self.batteryRemain
        self.num            = 0
        self.numidx         = np.zeros([N_VERTIPORT])
        self.numidx[vertiport] = 1
```

무인 이동체 제어 구현 (Environment Function)


```python
    def batteryConsump(self, action):
            batteryconsumption = 622 * 1000 * 60   # [J] Hovering
        elif action == 5:
            batteryconsumption = - self.maxbattery * 0.2 / 5
        else:
            batteryconsumption = 230 * 1000 * 60   # [J] Round-trip traveling

        self.batteryRemain -= batteryconsumption
        self.batteryRemain = min(self.batteryRemain, self.maxbattery)
        self.batteryRemain = max(self.batteryRemain, 0)
        if self.batteryRemain == 0: self.isworking = 0

    def _avail_action(self, userList, vertiportList):

        avail_action = np.ones(15)

        # Searching request action
        if self.x >=  GRID or self.z == H_MIN : avail_action[0] = 0
        if self.x <= -GRID or self.z == H_MIN : avail_action[1] = 0
        if self.y >=  GRID or self.z == H_MIN : avail_action[2] = 0
        if self.y <= -GRID or self.z == H_MIN : avail_action[3] = 0
        if self.x >= (GRID  / math.sqrt(2)) or self.y >= (GRID  / math.sqrt(2)) or self.z == H_MIN : avail_action[4] = 0
        if self.x <= (-GRID / math.sqrt(2)) or self.y >= (GRID  / math.sqrt(2)) or self.z == H_MIN : avail_action[5] = 0
        if self.x >= (GRID  / math.sqrt(2)) or self.y <= (-GRID / math.sqrt(2)) or self.z == H_MIN : avail_action[6] = 0
        if self.x <= (-GRID / math.sqrt(2)) or self.y <= (-GRID / math.sqrt(2)) or self.z == H_MIN : avail_action[7] = 0

        # Transportation action
        avail_action[8:] = 0 # 8 ~ 14

        if self.z == H_MAX:
            for v in range(N_VERTIPORT): # 10 11 12 13 14
                diff = math.sqrt((vertiportList[v].x - self.x)**2 + (vertiportList[v].y - self.y)**2)
                if diff <= VELOCITY: avail_action[v+10] = 1 # Vertiport에서 수직하강하는 action

            if self.vertiport != -1: avail_action[self.vertiport+10] = 0

        if self.z == H_MIN:
            if self.counter == 5:
                avail_action[8] = 1 # Vertiport에서 수직이륙하는 action
                avail_action[9] = 0 # waiting to charge at vertiport

            elif self.counter < 5:
                avail_action = np.zeros(15)
                avail_action[9] = 1 # waiting to charge at vertiport

        return avail_action
```

```python
    def arrive_process(self, userList, idx, id, vertiportList):
        userList[id].connect['isOnboard'] = 0
        userList[id].connect['isArrive'] = 1
        # 해당 Passenger를 내렸으므로, UAM의 상태 Update
        self.seat[idx] = -1
        self.takeoff += 1

        x_diff = vertiportList[userList[id].Departure].x - vertiportList[userList[id].Arrival].x
        y_diff = vertiportList[userList[id].Departure].y - vertiportList[userList[id].Arrival].y
        self.distance = math.sqrt((x_diff)**2 + (y_diff)**2)

    def onboard_process(self, user, idx):
        user.connect['drone'] = self.id
        user.connect['isOnboard'] = 1
        self.seat[idx] = user.id

    def transition(self, action, userList, vertiportList):

        if self.isworking == 1:
            if   action == 0: self.x += VELOCITY
            elif action == 1: self.x -= VELOCITY
            elif action == 2: self.y += VELOCITY
            elif action == 3: self.y -= VELOCITY
            elif action == 4:
                self.x += VELOCITY / math.sqrt(2)
                self.y += VELOCITY / math.sqrt(2)

            elif action == 5:
                self.x -= VELOCITY / math.sqrt(2)
                self.y += VELOCITY / math.sqrt(2)

            elif action == 6:
                self.x += VELOCITY / math.sqrt(2)
                self.y -= VELOCITY / math.sqrt(2)

            elif action == 7:
                self.x -= VELOCITY / math.sqrt(2)
                self.y -= VELOCITY / math.sqrt(2)

            elif action == 8: # Vertiport에서 수직이륙하는 action
                self.z = H_MAX
                self.counter = 0

            elif action == 9: # waiting to charge at vertiport
                self.counter += 1
```

무인 이동체 제어 구현 (Environment Function)


```python
            elif action == 8: # Vertiport에서 수직이륙하는 action
                self.z = H_MAX
                self.counter = 0

            elif action == 9: # waiting to charge at vertiport
                self.counter += 1

                empty_mask = (self.seat == -1)
                empty_idx = [i for i, x in enumerate(empty_mask) if x ]
                for i in range(len(empty_idx)):
                    idx = empty_idx[i]
                    for user in userList:
                        if (user.connect['isArrive'] == 0) and (user.connect['isOnboard'] == 0) and (user.Departure == self.vertiport):
                            self.onboard_process(user, idx)
                            break

            elif action >= 10: # 수직착륙하는 action
                self.vertiport = action - 10
                self.numidx[self.vertiport] += 1
                self.x, self.y, self.z = vertiportList[self.vertiport].x, vertiportList[self.vertiport].y, H_MIN

                seat_mask = (self.seat != -1)
                seat_idx = [i for i, x in enumerate(seat_mask) if x]

                # 내릴 Vertiport 결정된 이후의 Transition
                for i in range(len(seat_idx)):  # 탑승한 Passenger 존재
                    idx = seat_idx[i]           # 탑승한 좌석 위치
                    id = self.seat[idx]         # 탑승한 Passenger id
                    if (userList[id].Arrival == vertiportList[self.vertiport].id):
                        self.arrive_process(userList, idx, id, vertiportList) # 해당 Passenger를 내렸으므로, UAM의 상태 Update

                empty_mask = (self.seat == -1)
                empty_idx = [i for i, x in enumerate(empty_mask) if x]

                for i in range(len(empty_idx)):
                    idx = empty_idx[i]

                    for user in userList:
                        if (user.connect['isArrive'] == 0) and (user.connect['isOnboard'] == 0) and (user.Departure == self.vertiport):
                            self.onboard_process(user, idx)
                            break

        # state space 맞추어주기.
        self.x = self.clamp(self.x, -GRID, GRID)
        self.y = self.clamp(self.y, -GRID, GRID)
        self.batteryConsump(action)
```

무인 이동체 제어 구현 (Environment Function)


```python
class Utility:

    def __init__(self):
        self.T_service_rate     = 0
        self.T_onboard_rate     = 0
        self.T_service_user     = 0
        self.T_onboard_user     = 0
        self.grid               = 60
        self.scale              = self.grid / GRID
        self.I_reward           = np.zeros(N_COMMAGENT+N_DQNAGENT)
        self.I_battery          = np.zeros(N_COMMAGENT+N_DQNAGENT)
        self.I_onboard_user     = np.zeros(N_COMMAGENT+N_DQNAGENT)
        self.I_service_user     = np.zeros(N_COMMAGENT+N_DQNAGENT)
        self.I_onboard_distance = np.zeros(N_COMMAGENT+N_DQNAGENT)
        self.SERVICED           = np.zeros(N_COMMAGENT+N_DQNAGENT+1)
        self.num_Takeoff        = np.zeros([N_VERTIPORT])

    def get_utils_info(self):

        return self.T_service_rate, self.I_battery

    def _calculate_support(self, userList, agentList, vertiportList):

        for i in range(N_USER):
            droneId, isOnboard, isArrive = userList[i].is_support()
            # print('droneID {} isOnboard {} isArrive {}'.format(droneId, isOnboard, isArrive))
            self.T_service_user += isArrive
            self.T_onboard_user += isOnboard
            if droneId != -1:
                self.I_service_user[droneId] += isArrive
                self.I_onboard_user[droneId] += isOnboard

        for j, agent in enumerate(agentList):
            seat_mask = (agent.seat != -1)
            seat_idx  = [a for a, b in enumerate(seat_mask) if b]

            total_distance = 0
            for i in range(len(seat_idx)): # 탑승한 Passenger 존재
                idx = seat_idx[i] # 탑승한 좌석 위치
                id = agent.seat[idx] # 탑승한 Passenger id

                x_diff          = agent.x - vertiportList[userList[id].Arrival].x
                y_diff          = agent.y - vertiportList[userList[id].Arrival].y
                total_distance += math.sqrt((x_diff)**2 + (y_diff)**2)

            if len(seat_idx) == 0:  self.I_onboard_distance[j] = total_distance / (GRID)
            else:                   self.I_onboard_distance[j] = total_distance / ( len(seat_idx) * GRID )
```

무인 이동체 제어 구현 (Environment Function)


```python
    def _calculate_energy_consumption(self, agentList):

        for drone in agentList:
            self.I_battery[drone.id]  = drone.batteryRemain

    def _calculate_indiv_reward(self, agentList, vertiportList):

        for i, drone in enumerate (agentList):
            takeoff_mask = (agentList[i].numidx > 0)
            takeoff_idx  = [m for m, n in enumerate(takeoff_mask) if n]

            self.I_reward[i] = (agentList[i].takeoff + len(takeoff_idx) - self.I_onboard_distance[i] + self.I_battery[i] / drone.maxbattery) # type 1

        return self.I_reward

    def _calculate_common_reward(self, userList, agentList, vertiportList):

        for i, agent in enumerate(agentList):
            self.num_Takeoff += agent.numidx

        takeoff_mask  = (self.num_Takeoff > 0)
        takeoff_idx   = [i for i, x in enumerate(takeoff_mask) if x]

        Common_Reward = np.full((N_COMMAGENT+N_DQNAGENT), self.T_service_user / (N_DQNAGENT + N_COMMAGENT))

        return Common_Reward

    def calculate_reward(self, userList, agentList, vertiportList):

        self.__init__() # 초기화 먼저 해줌
        self._calculate_support(userList, agentList, vertiportList)
        self._calculate_energy_consumption(agentList)
        self.SERVICED[0]                          = self.T_service_user
        self.SERVICED[1:(N_COMMAGENT+N_DQNAGENT+1)] = np.copy(self.I_service_user)
        Common_Reward                             = self._calculate_common_reward(userList, agentList, vertiportList)
        Individual_Reward                         = self._calculate_indiv_reward(agentList, vertiportList)
        Reward                                    = np.array(Individual_Reward) + Common_Reward

        return Reward / 100.0
```

무인 이동체 제어 구현 (Environment Function)


```python
class Environment:

    def __init__(self):

        # [1] Environment
        self.EPLEN        = EP_LEN
        self.numVertiport = N_VERTIPORT
        self.numUser      = N_USER
        self.numCommAgent = N_COMMAGENT
        self.numDQNAgent  = N_DQNAGENT
        self.numAgent     = self.numCommAgent + self.numDQNAgent
        self.GridRadius   = GRID
        self.hMax         = H_MAX
        self.hMin         = H_MIN
        self.t            = 0
        self.numTakeoff   = np.zeros([self.numAgent, N_VERTIPORT])

        # [2] Environment --> RLAgent
        self.o            = None
        self.o_prime      = None

        # [3] Environment Initialization
        self.Initialize()

    def reset(self):
        self.Initialize()

    def get_available_action(self):
        available_action = []

        for i in range(self.numAgent):
            available_action.append(self.agentList[i]._avail_action(self.userList, self.vertiportList))
        return np.array(available_action)

    def Initialize(self):
        vertiportList = self._initVertiport()
        self._initUser()
        self._initAgent(vertiportList)

        self.utility = Utility()
        self.common_obs = self.getCommonObs()
        # self.common_obs_prime = np.copy(self.common_obs)
        self.o = self.getagentObs(self.t)
        self.o_prime = np.copy(self.o)
        self.ava_prime = self.get_available_action()
```

```python
    def get_info(self):
        info = dict()
        info['episode_limit'] = self.EPLEN
        info['n_Comm_agents'] = self.numCommAgent
        info['n_DQN_agents']  = self.numDQNAgent
        info['n_agents']      = self.numCommAgent + self.numDQNAgent
        info['n_actions']     = 15
        info['obs_dim']       = self.o.shape[-1]
        return info


    def _initVertiport(self):
        self.vertiportList = []
        for i in range(self.numVertiport):
            if  i == 0:
                x, y = 0, 0 # [m]
                self.vertiportList.append(Vertiport(id=i, x=x, y=y, z=H_MIN))
            elif i == 1:
                x, y = -26192 , -17445
                self.vertiportList.append(Vertiport(id=i, x=x, y=y, z=H_MIN))
            elif i == 2:
                x, y = 19937, 22918
                self.vertiportList.append(Vertiport(id=i, x=x, y=y, z=H_MIN))
            elif i == 3:
                x, y = 21794, -13731
                self.vertiportList.append(Vertiport(id=i, x=x, y=y, z=H_MIN))
            elif i == 4:
                x, y = 17396, -7183
                self.vertiportList.append(Vertiport(id=i, x=x, y=y, z=H_MIN))

        return self.vertiportList

    def _initUser(self):
        self.userList = []
        for i in range(self.numUser):
            Departure = np.random.randint(0, self.numVertiport) # 1, 2, 3, 4번 Vertiport
            Arrival = np.random.randint(0, self.numVertiport)

            while(Departure == Arrival):
                Departure = np.random.randint(0, self.numVertiport)
                Arrival = np.random.randint(0, self.numVertiport)
            self.userList.append(User(id=i, Departure=Departure, Arrival=Arrival))
```

무인 이동체 제어 구현 (Environment Function)


```python
    def _initAgent(self, vertiportList):
        self.agentList = []
        marker = np.zeros([self.numVertiport], dtype=int)
        for i in range(self.numAgent):
            idx = np.random.randint(0, self.numVertiport)
            while marker[idx] > 1: # 0, 1
                idx = np.random.randint(0, self.numVertiport)
            marker[idx] += 1
            self.agentList.append(Drone(id=i, x=vertiportList[idx].x, y=vertiportList[idx].y, z=vertiportList[idx].z, vertiport=vertiportList[idx].id))


    def plot(self):
        fig, ax = plt.subplots(figsize=(4,4),dpi=150)
        DX, DY, DR = [],[],[]
        AX, AY, AR = [],[],[]
        CX, CY, CR = [],[],[]
        OX, OY = [],[]

        for i in range(self.numVertiport):
            DX.append(self.vertiportList[i].x)
            DY.append(self.vertiportList[i].y)

        for i in range(N_COMMAGENT):
            CX.append(self.agentList[i].x)
            CY.append(self.agentList[i].y)

        for i in range(N_COMMAGENT, self.numAgent):
            AX.append(self.agentList[i].x)
            AY.append(self.agentList[i].y)

        if len(DX):
            ax.scatter(DX, DY, s  = 90, c = 'k', marker = 'o')

        if len(CX):
            ax.scatter(CX, CY, s  = 90, c = 'magenta', marker = '*')


        if len(AX):
            ax.scatter(AX, AY, s  = 90, c = 'dodgerblue', marker = '*')

        if len(OX):
            ax.scatter(OX, OY, s  = 90, c = 'orangered', marker = 'X')
```

무인 이동체 제어 구현 (Environment Function)


```python
        ax.grid(which='major', axis='both', linestyle='-', color='k', linewidth=1)
        ax.set_xlabel('x [m]')
        ax.set_ylabel('y [m]')
        plt.xlim([-self.GridRadius, self.GridRadius])
        plt.ylim([-self.GridRadius, self.GridRadius])
        plt.draw()
        buf = io.BytesIO()
        plt.savefig(buf, format='jpeg')
        buf.seek(0)
        image = PIL.Image.open(buf)
        plt.close()
        image = ToTensor()(image).unsqueeze(0)[0]
        return image

    def getCommonObs(self):


        vertiport_pos = np.array([[self.vertiportList[k].x, self.vertiportList[k].y] for k in range(self.numVertiport)]).flatten()

        Position = np.hstack([vertiport_pos])
        ## Normalize
        Position = Position / self.GridRadius

        # [3] Utility 정보
        Util          = np.array([self.utility.T_service_rate, self.utility.T_onboard_rate])
        Util          = np.append(Util, np.array([self.utility.I_service_user, self.utility.I_onboard_user, self.utility.I_onboard_distance]).flatten())

        CommonObs = np.hstack([Position, Util])
        return CommonObs

    def getagentObs(self, t):

        Obs = []
        obs_common = self.common_obs
```

무인 이동체 제어 구현 (Environment Function)


```python
472         for i in range(self.numAgent):
473             # o_partial = []
474
475         # [1] 자신의 정보
476
477             # agent = self.agentList[i] # 현재 주체가 되는 Agent의 index == i
478             o_pos = []
479             o_pos.append(self.agentList[i].x / self.GridRadius)
480             o_pos.append(self.agentList[i].y / self.GridRadius)
481             o_pos.append(self.agentList[i].z / self.GridRadius)
482             o_pos.append(self.agentList[i].batteryRemain / self.agentList[i].maxbattery)
483             # o_pos.append(self.agentList[i].batteryRemain)
484
485         # [2] 다른 Agent의 위치 정보
486
487             o_pos_agent = []
488             for k in range(self.numAgent):
489                 x_diff = self.agentList[i].x - self.agentList[k].x
490                 y_diff = self.agentList[i].y - self.agentList[k].y
491                 diff   = math.sqrt((x_diff)**2 + (y_diff)**2)
492
493                 if diff <= OBSERVABLE:
494                     o_pos_agent.append(self.agentList[k].x / self.GridRadius)
495                     o_pos_agent.append(self.agentList[k].y / self.GridRadius)
496                     # o_pos_agent.append(self.agentList[k].z / self.GridRadius)
497                     o_pos_agent.append(diff / self.GridRadius)
498
499                 else:
500                     o_pos_agent.append(-1)
501                     o_pos_agent.append(-1)
502                     # o_pos_agent.append(-1)
503                     o_pos_agent.append(-1)
504
505         # [3] Onloaded Passenger의 정보
506
507             o_seat = []
508
509             seat_mask = (self.agentList[i].seat != -1)
510             seat_idx = [j for j, x in enumerate(seat_mask) if x]
511
512             seat_counter = 0
513             for k in range(MTOW):
514                 if seat_mask[k]:
```

```python
513                 for k in range(MTOW):
514                     if seat_mask[k]:
515                         idx = seat_idx[seat_counter]
516                         seat_counter += 1
517                         id = self.agentList[i].seat[idx]
518
519                         # o_seat.append(self.userList[id].connect['drone'] / self.numAgent)
520                         # o_seat.append(self.userList[id].connect['isOnboard'])
521                         # o_seat.append(self.userList[id].connect['isArrive'])
522                         o_seat.append(self.userList[id].Arrival / N_VERTIPORT)
523                         o_seat.append(self.userList[id].Departure / N_VERTIPORT)
524
525                     else:
526                         # o_seat.append(-1)
527                         o_seat.append(-1)
528                         o_seat.append(-1)
529
530         # [4] Vertiport의 위치 정보
531
532             o_pos_vertiport = []
533             for k in range(self.numVertiport):
534                 x_diff = self.agentList[i].x - self.vertiportList[k].x
535                 y_diff = self.agentList[i].y - self.vertiportList[k].y
536                 diff   = math.sqrt((x_diff)**2 + (y_diff)**2)
537
538                 if diff <= OBSERVABLE:
539                     o_pos_vertiport.append(self.vertiportList[k].x / self.GridRadius)
540                     o_pos_vertiport.append(self.vertiportList[k].y / self.GridRadius)
541                     # o_pos_vertiport.append(self.vertiportList[k].z / self.GridRadius)
542                     o_pos_vertiport.append(diff / self.GridRadius)
543
544                 else:
545                     o_pos_vertiport.append(-1)
546                     o_pos_vertiport.append(-1)
547                     # o_pos_vertiport.append(-1)
548                     o_pos_vertiport.append(-1)
549
550             o_pos           = np.array(o_pos)
551             o_pos_agent     = np.array(o_pos_agent)
552             o_pos_vertiport = np.array(o_pos_vertiport)
553             o_seat          = np.array(o_seat)
554
```

무인 이동체 제어 구현 (Environment Function)


```python
            o_idx = np.zeros(self.numAgent)
            o_idx[self.agentList[i].id] += 1
            o_partial = np.hstack([o_pos, o_pos_agent, o_pos_vertiport, o_seat]) # 모든 정보 Concatenation
            # o = np.hstack([o_idx, obs_common, o_partial])
            # o = np.hstack([t/EP_LEN, o_idx, o_partial])
            o = np.hstack([o_idx, o_partial])
            Obs.append(np.copy(o))
        Obs = np.array(Obs)
        return Obs


    def step(self, actions, time, epoch, today, info):
        self.t = time
        # print(actions)
        self.o = np.copy(self.o_prime)
        for i in range(self.numAgent):
            self.agentList[i].transition(actions[i], self.userList, self.vertiportList)
        # self.getCommonObs()
        self.o_prime = self.getagentObs(time)

        rewards = self.utility.calculate_reward(self.userList, self.agentList, self.vertiportList)

        if(time == 0):
            self.x_COMM = np.zeros([EP_LEN, N_COMMAGENT])
            self.y_COMM = np.zeros([EP_LEN, N_COMMAGENT])
            self.z_COMM = np.zeros([EP_LEN, N_COMMAGENT])
            self.x_DNN  = np.zeros([EP_LEN, N_DQNAGENT])
            self.y_DNN  = np.zeros([EP_LEN, N_DQNAGENT])
            self.z_DNN  = np.zeros([EP_LEN, N_DQNAGENT])

        if(time == EP_LEN):
            for i, agent in enumerate(self.agentList):
                self.numTakeoff[i] = agent.numidx

        self.ava_prime = self.get_available_action()
```

무인 이동체 제어 구현 (Reply Buffer Function)


```python
import numpy as np
import threading
import collections

def get_experience(o,actions, rewards, o_prime):
    experience = {
            'o': o,
            'a' : actions,
            'rewards': rewards,
            'o_prime': o_prime,
        }
    return experience
class ReplayBuffer:
    def __init__(self, args):
        self.args = args
        self.buffer = []

    def push(self, experience):
        self.buffer.append(experience)
```

```python
    def sample(self):
        O,ACTIONS, REWARDS, O_PRIME = [],[],[],[]
        for i in range(len(self.buffer)):
            O.append(self.buffer[i]['o'])
            ACTIONS.append(self.buffer[i]['a'])
            REWARDS.append(self.buffer[i]['rewards'])
            O_PRIME.append(self.buffer[i]['o_prime'])
        O = np.array(O)
        ACTIONS = np.array(ACTIONS)
        REWARDS = np.array(REWARDS)
        O_PRIME = np.array(O_PRIME)

        samples = dict({
            'O' : O,
            'A' : ACTIONS,
            'REWARDS' : REWARDS,
            'O_PRIME' : O_PRIME
        })
        self.buffer = []
        return samples

    def size(self):
        return len(self.buffer)
```

Thank you for your attention!

- More questions?
  - joongheon@korea.ac.kr (Advisor)
  - kingdom0545@korea.ac.kr (Presenter)